Name - Shubh Dwivedi Internship Work Submitted to:
Section - DS-2 Dr. Amita Saroj Mam
Uni. Roll no. - 2023932

Q1 (a). Explain the various layers of OS and discuss the advantages and disadvantages of different OS structure in detail.

There are 5 layers in Operating System -

1. Hardware layer -

This layer consists of the physical components like CPU, memory, I/O devices.

2. Kernel layer -

This is the core of OS. It handles process management, memory management, device management.

3. System Call Interface

It provides communication between user programs and the OS.

4. System Programs

It consists of compilers, editors, loaders, utilies.

5. User Applications -

In this layer, there are browsers, MS word, media players, etc.

OS structure -

1. Monolithic Structure -

Ex: Unix

Advantage -

- Fast execution
- Efficient communication

Disadvantages -

- Difficult to maintain
- One error may ~~do~~ crash the entire system.

2. Layered Structure -

Advantages -

- Easy debugging
- Better modularity

Disadvantages -

- Reduced performance
- Difficult layer design

3. Microkernel -

Advantages -

- Reliable and secure
- Easy to extend

Disadvantages -

- More context switching
- Lower performance.

4. Modular Structure -

Advantages -

- Flexible
- Easy to update

Disadvantages -

- Complex design.

(6). What are System Calls? Explain the types of system calls used in operating systems with examples.

System calls are the interface through which a user program communicates with the operating system. Whenever an application needs a service such as file access, process creation, memory allocation or device communication, it requests the operating system through a system call. These calls provide controlled access to hardware resources and ensure system security.

Types of system calls -

1. Process control -

It is used for process creation and termination.

Examples :

- fork()
- exit()
- wait()

2. File Management -

It is used to create and manipulate files.

Examples :

- open()
- read()
- write()
- close()

3. Device Management -

It is used for device operations.

Examples - ioctl(), read(), write().

4. Information Maintenance -

Used to obtain the system information.

Examples - getpid(), time().

5. Communication -

Used for the inter-process communication.

Examples - pipe(), send(), receive().

6. Protection -

Controls access permissions.

Examples - chmod(), setuid().

Q2 (b). Answer -

Critical Section Problem -

A critical section is a part of a program where shared data is accessed.

Structure -

Entry section
Critical Section
Exit section
Remainder Section

Requirements -

1. Mutual Exclusion -

only one process enters at a time.

2. Progress -

Processes should not wait unnecessarily.

3. Bounded Waiting -

A process must get a chance within finite time.

## CPU Scheduling Criteria -

1. CPU Utilization -

Keep CPU busy.

2. Throughput -

Maximum processes completed.

3. Turnaround Time -

Completion Time - Arrival Time

4. Waiting Time -

Time spent in ready queue.

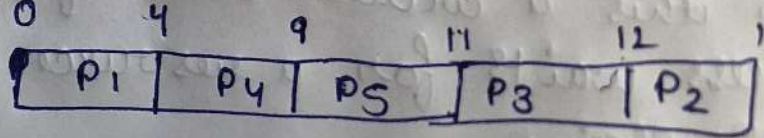
5. Response Time -

Time from request to first response.

(C).

| Process | AT | BT | Priority | CT | TAT | WT |
|---|---|---|---|---|---|---|
| P1 | 0 | 4 | 2 | 4 | 4 | 0 |
| P2 | 1 | 3 | 3 | 9 | 14 | 11 |
| P3 | 2 | 1 | 4 | 11 | 10 | 9 |
| P4 | 3 | 5 | 5 | 12 | 6 | 1 |
| P5 | 4 | 2 | 5 | 15 | 7 | 5 |

Gantt Chart -




TAT - CT - AT

WT - TAT - BT

Average Waiting Time –

$$AWT = \frac{0+11+9+1+5}{5} = 5.2$$

Average Turnaround Time –

$$ATAT = \frac{1+14+10+6+7}{5} = 8.2$$

Q3. (b) Answer-

A deadlock is a situation in which two or more processes are permanently blocked because each process is waiting for a resource held by another process.

For a deadlock, there are four necessary conditions-

1. Mutual Exclusion -

It means at least one resource can be used by only one process at a time.

2. Hold and wait -

In this, process holding one resource waits for additional resources.

3. No Preemption -

It means that resources cannot be forcibly taken from a process until it releases them voluntarily.

4. Circular Wait -

In this a circular chain of processes exists and each process waits for a resource held by the next process in the chain.

(C)

Given Data -

Allocation Matrix -

| Process | A | B | C |
|---|---|---|---|
| P0 | 1 | 1 | 2 |
| P1 | 2 | 1 | 2 |
| P2 | 4 | 0 | 1 |
| P3 | 0 | 2 | 2 |
| P4 | 1 | 1 | 2 |

Max Matrix -

| Process | A | B | C |
|---|---|---|---|
| P0 | 4 | 3 | 3 |
| P1 | 3 | 2 | 2 |
| P2 | 9 | 6 | 2 |
| P3 | 7 | 5 | 3 |
| P4 | 1 | 1 | 2 |

Available -

| A | B | C |
|---|---|---|
| 2 | 1 | 0 |

Need Matrix - (Need = Max - Alloc$^n$)

| Process | A | B | C |
|---|---|---|---|
| P0 | 3 | 2 | 1 |
| P1 | 1 | 1 | 0 |
| P2 | 5 | 0 | 1 |
| P3 | 7 | 3 | 1 |
| P4 | 0 | 0 | 0 |

fe Sequence

$P_4 \rightarrow P_1 \rightarrow P_0 \rightarrow P_2 \rightarrow P_3$

Q 4.(b). Answer -

File allocation methods determine how files are stored on secondary storage devices such as hard disks. The three major file allocation techniques are ~~co~~ti-contiguous allocation, linked allocation and indexed allocation.

In contiguous allocation, a file occupies consecutive disk blocks. The operating system stores the starting block address and the length of the file.

In linked allocation, file blocks are stored anywhere on the disk and linked together using pointers. Each block contains a pointer to the next block in the file.

In indexed allocation, file blocks are stored in index block which is ~~cre~~ created for each file. The index ~~conta~~ block contains the addresses of all file blocks.

(c). Given -

Request Queue -

82, 170, 43, 140, 24, 16, 190

Initial Head Position - 50

Range - 0 to 199

## FCFS –

Head Movement :

50 → 82 → 170 → 43 → 140 → 24 → 16 → 190

Total Seek Distance:

32 + 88 + 127 + 97 + 116 + 8 + 174 = 642 cylinders

## SSTF –

Sequence:

50 → 43 → 24 → 16 → 82 → 140 → 170 → 190

~~Tot~~ TSD :

7 + 19 + 8 + 66 + 58 + 30 + 20 = 208 cylinders

## SCAN –

50 → 43 → 24 → 16 → 0 → 82 → 140 → 170 → 190

TSD:

7 + 19 + 8 + 16 + 82 + 58 + 30 + 20 = 240 cylinders

## CSCAN –

50 → 43 → 24 → 16 → 0 → 199 → 190 → 170 → 140 → 82

TSD:

50 + 199 + 9 + 20 + 30 + 58 = 366 cylinders

## LOOK –

50 → 43 → 24 → 16 → 82 → 140 → 170 → 190

TSD:

7 + 19 + 8 + 66 + 58 + 30 + 20 = 208 cylinders

## CLOOK –

50 → 43 → 24 → 16 → 190 → 170 → 140 → 82

TSD:

7 + 19 + 8 + 174 + 20 + 30 + 58 = 316 cylinders

Q5. (b). Answer :

```
#!/bin/bash
    echo "Enter first string"
    read str1
    echo "Enter second string"
    read str2

concat = $str1 $str2
    echo "Concated string : $concat"
len = ${#concat}
    echo "Length of string : $len"
sub = ${concat :0 :5}
echo "Substring (first 5 characters) : $sub"
```

(c).

Linux uses a modular monolithic kernel architecture. Like traditional monolithic kernels, core operating system services such as process management, memory management, file systems, and device drivers execute in kernel space. However Linux differs by supporting dynamically loadable kernel modules. These modules can be added or removed without recompiling or restarting the kernel. This approach provides flexibility, easier maintenance, and better performance compared to purely monolithic systems.

Modular linux systems use the Completely Fair Scheduler (CFS) for handling multitasking efficiently.